

OpenAI Whisper

The architecture and decoding of a weakly-supervised sequence-to-sequence speech recogniser

A conceptual walkthrough — grounded in the `voice` toolkit's pure-Rust reimplementation

Abstract. — Whisper turns a recording of speech into text. It does so not by classifying sounds into phonemes but by **generating a sentence, one token at a time, conditioned on audio** — it is a language model that happens to be able to hear. That single framing explains almost everything about the model: why it punctuates and capitalises for free, why a four-token prefix can switch it between transcribing and translating, why it needs a key/value cache to run at a sensible speed, and why, handed thirty seconds of silence, it will confidently invent a sentence. This document walks the whole path — log-mel front-end, asymmetric encoder/decoder, the special-token protocol, cached greedy decoding — and then spends real time on the two things a maintainer actually needs: the causal-mask trap that once shipped a port loading at 100% weight coverage and producing fluent nonsense, and how a port is verified beyond coverage at all.

Contents

1	What speech recognition actually is	2
2	Background: a few building blocks	3
3	The front-end: from waveform to log-mel	4
3.1	The mel warp	4
3.2	Compression, and the cost of normalising per window	5
4	The encoder: thirty seconds of audio, once	5
5	The decoder, and the asymmetry that defines large-v3-turbo	6
5.1	32 against 4	7
5.2	A note on dimensions (large-v3-turbo)	7
5.3	Two details that only bite on a port	8
6	The special-token protocol: steering, not configuring	8
7	Decoding: greedy search with a key/value cache	10
7.1	Why a cache at all	10
7.2	Where the cache lives, and why	10
7.3	Bounding the loop	11
8	The causal-mask trap	11
8.1	What happened	11
8.2	Why it was silent, twice over	12
8.3	And why coverage did not catch it	12
9	Two runtimes, one loop	12
9.1	The ONNX path is a deployment target, not a legacy	13
10	Segmentation, and hallucination on silence	13
10.1	Reusing the sentence-safe slicer	14
11	Verifying the port	14

1 What speech recognition actually is

Automatic speech recognition (**ASR**) is the problem of mapping a waveform x — a long list of numbers — to the sequence of words y that produced it. Classically this was factored into pieces: an **acoustic model** scoring which phoneme each frame sounds like, a **pronunciation lexicon** spelling words out as phonemes, and a **language model** saying which word sequences are plausible. Each piece was trained separately, and gluing them together was most of the work.

Whisper throws the factorisation away. It is a single **sequence-to-sequence transformer** trained to maximise the likelihood of the reference transcript given the audio, and nothing else:

$$L = - \sum_t \log p_\theta(y_t \mid y_{<t}, \text{audio}).$$

That is exactly the training objective of a language model, with one extra thing in the conditioning bar. Read it left to right and the consequence is unavoidable: at every step the model is asked “**given the audio, and given the text so far, what word comes next?**” — and the second half of that question is answerable without the audio at all.

Three properties follow, and they are the whole personality of the model.

Formatting comes free casing, punctuation, digits written as numerals — all of it is just **text the model learned to produce**, not a post-processing stage bolted on afterwards. There is no lexicon to extend and no phoneme inventory to curate.

Multilinguality comes free the same decoder emits Japanese kana and Latin letters from the same vocabulary, because the training corpus — 680 000 hours scraped from the web with **weak** supervision, i.e. transcripts of unaudited quality — contained both. This is also why forcing the language (§6) helps on short clips: detection is a guess the model makes from a fraction of a second of audio.

Hallucination comes free too and this is the price of the other two. A language model asked to continue when the audio says nothing will continue **anyway**, fluently, because fluent continuation is the only thing it was ever trained to do (§10).

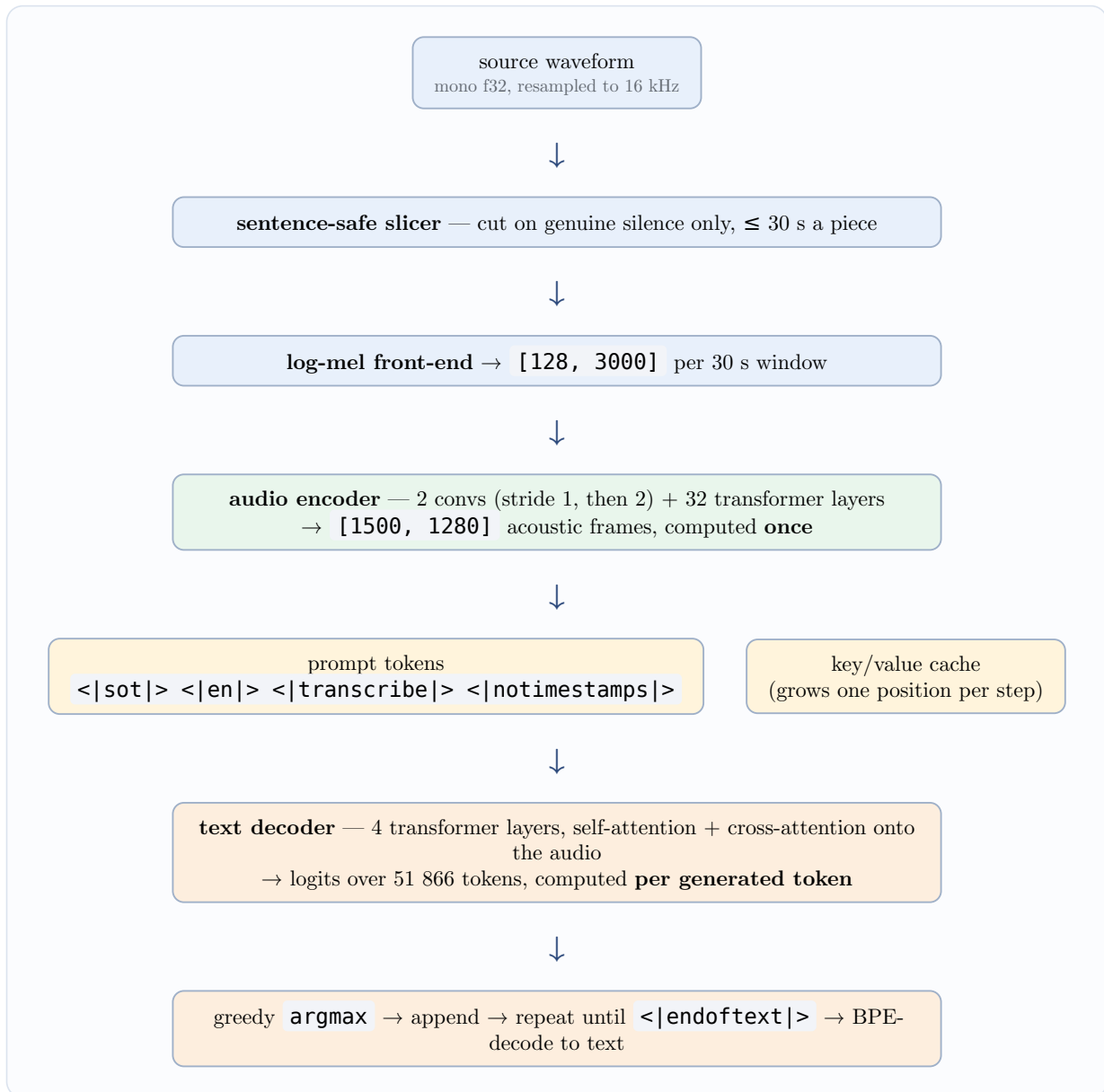


Figure 1: The end-to-end recognition path. Everything left of the decoder is fixed-cost signal analysis; the decoder is the autoregressive loop, and it is the only part that runs once per output token.

The rest of this document unfolds that diagram top to bottom, then turns to the two maintenance topics — the causal mask, and how the port is checked.

2 Background: a few building blocks

Readers new to speech machine learning may want these six ideas first; every later section leans on them. Experienced readers can skip ahead.

Digital audio — samples & sample rate a sound wave is stored as a long list of numbers, the **samples**, measured many thousands of times per second. The **sample rate** is how many per second. Whisper analyses at 16 kHz — enough for speech content, far below music fidelity — and everything upstream must resample to it (`stt_core::SAMPLE_RATE`).

Spectrogram (STFT) the raw sample list is hard to reason about directly. Sliding a short window along the signal and taking a **Fourier transform** of each window gives the **short-time Fourier transform** — a 2-D picture of **how much energy sits at each frequency, moment by moment**. Columns are time frames; rows are frequency bins.

The mel scale human hearing resolves low frequencies far more finely than high ones. The **mel scale** warps frequency to match, so equal distances on it are roughly equal perceptual distances. A **mel spectrogram** regroups the STFT’s frequency rows into a smaller set of perceptually spaced **mel bands** (128 of them here). Whisper’s front-end is a mel spectrogram and nothing more (§3).

Tokens & BPE text is fed to the model as integer ids, not characters. **Byte pair encoding** builds a vocabulary of common byte sequences — “ the ” is one token, a rare word is several — so a sentence becomes a short sequence of ids drawn from a fixed table (51 866 entries here). The same table carries **control tokens** that are never printed (§6).

Attention & the transformer **attention** lets every position look at every other and pull in what it needs, weighted by relevance. Written out, with queries Q , keys K and values V all obtained by linear projection:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{head}}}} + M\right)V.$$

The $\sqrt{d_{\text{head}}}$ keeps the dot products in a sane range as width grows; M is an additive **mask** of 0 and $-\infty$ that forbids some query–key pairs, and is where §8’s trap lives. A **transformer** stacks blocks of attention plus a small feed-forward network, each wrapped in a residual connection.

Autoregressive decoding the decoder emits one token, appends it to its own input, and runs again — so generating T tokens means T forward passes. **Greedy** decoding takes the highest-scoring token at every step. This loop is why the cache of §7 matters so much and why the decoder’s depth is worth far more than the encoder’s (§5).

3 The front-end: from waveform to log-mel

The encoder never sees a waveform. It sees a [128, 3000] picture of one — 128 mel bands by 3000 frames — and the numbers producing that picture are not negotiable, because the encoder was trained on precisely these numbers. **stt_core**’s **LogMel** reproduces the reference front-end exactly; getting any constant wrong degrades recognition without ever raising an error.

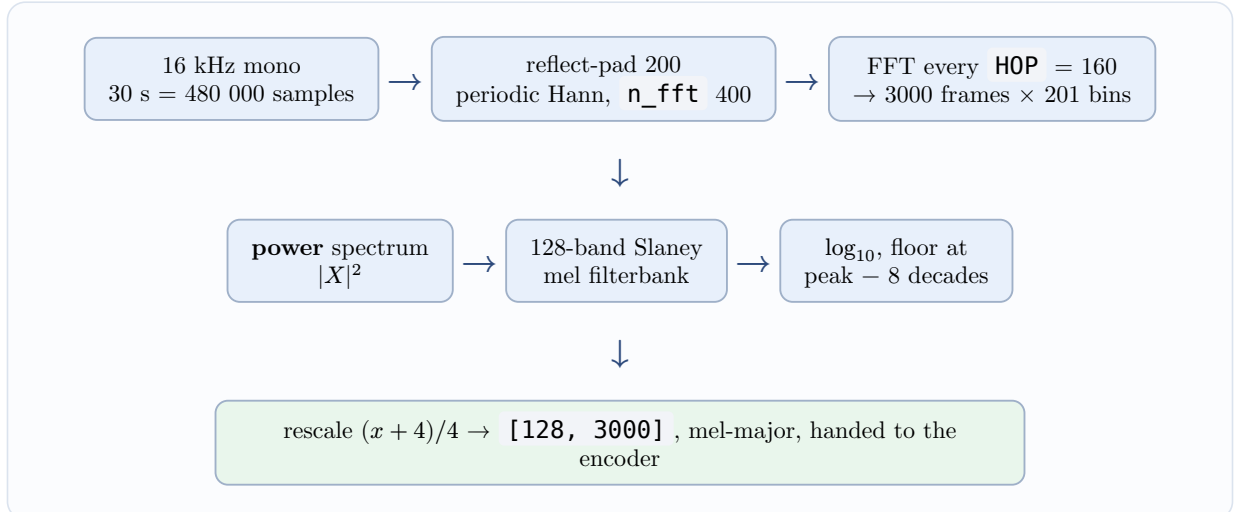


Figure 2: The log-mel front-end. Every constant is fixed by what the encoder was trained on; the only free parameter is **n_mels**, and it is read from the checkpoint rather than assumed.

3.1 The mel warp

Whisper uses librosa’s **Slaney** mel scale — linear below 1 kHz, logarithmic above — not the single-formula HTK version that appears in most textbooks (and that **rvc_core**’s RMVPE front-end uses). With $f_{\text{sp}} = 200/3$ and $\rho = \ln(6.4)/27$:

$$\text{mel}(f) = \begin{cases} f/f_{\text{sp}} & f < 1000 \text{ Hz} \\ 15 + \ln(f/1000)/\rho & f \geq 1000 \text{ Hz} \end{cases}$$

128 triangular filters are laid out at equal mel spacing between 0 Hz and Nyquist (8 kHz), each normalised by $2/(\text{hi} - \text{lo})$ so it integrates to a constant rather than peaking at one — Slaney **area** normalisation. This is `build_slaney_mel`, and it reproduces the filterbank Whisper ships as a precomputed `.npz` file, which is why the toolkit needs no asset for it.

3.2 Compression, and the cost of normalising per window

Raw mel energies span many orders of magnitude, so they are compressed logarithmically and then clamped:

$$X = \log_{10} \max(M, 10^{-10}), \quad X' = (\max(X, \max(X) - 8) + 4)/4.$$

Two separate things happen in that second expression. The **clamp** discards anything more than eight decades (80 dB) below the loudest bin **in this window** — dynamic range the model cannot use. The **rescale** then maps the surviving range into roughly unit scale.

The consequence worth internalising: because the floor is taken against $\max(X)$ over the whole window, the output always spans **exactly 2**, but **where that span sits** follows the window’s loudest bin. It is therefore not bounded to $[-1, 1]$, and — more importantly — **this is not a streaming transform**. A 30 s window must be complete before any of its frames are final, which is precisely why `stt` reads all of stdin before transcribing anything while `rvc serve` is a true streaming filter. It is also a real cost: a window containing one loud cough and one whispered sentence normalises against the cough, pushing the whisper eight decades down towards the floor. Segmenting first (§10) keeps loud and quiet material in different windows and so sidesteps it.

A short clip is **zero-padded** to the full 480 000 samples rather than analysed short (`LogMel::compute`). That is not an accommodation — the encoder’s positional table is exactly 1500 entries wide and the model was trained on padded windows, so a full-width window is what it expects. The unit tests `a_full_window_is_exactly_the_encoder_width`, `short_input_is_padded_not_truncated` and `the_dynamic_range_is_always_eight_decades` pin all three behaviours.

4 The encoder: thirty seconds of audio, once

The `AudioEncoder` consumes a log-mel tensor shaped `[batch, n_mels, frames]` and returns acoustic features shaped `[batch, frames/2, d_model]`. Two things happen to the mel picture before the transformer stack ever sees it.

A two-convolution stem `conv1` is a `Conv1d` with kernel 3, stride 1, mapping 128 mel bands to the 1280-wide residual stream; `conv2` has kernel 3 and **stride 2**. Both are followed by `gelu`. The stride is the whole point: 3000 mel frames (one per 10 ms) become **1500 encoder frames** (one per 20 ms), halving the sequence length before any quadratic attention runs. Get it wrong and every positional embedding lines up against the wrong moment in time — a failure no weight-coverage check would notice, which is why `encoder_halves_the_frame_rate` asserts the output shape directly.

Sinusoidal positions the encoder’s `embed_positions` table is **sinusoidal** by construction, but it is **stored in the checkpoint as a plain table** and therefore loaded rather than recomputed. Fixed positions are the right choice here because audio frames are uniformly spaced in time and there is no vocabulary to learn against; the decoder’s positions, over tokens of wildly varying duration, are learned instead (§5).

Then 32 identical `EncoderLayer` s. Each is a **pre-norm** residual block — normalise, transform, add:

```
x = x + self_attn(self_attn_layer_norm(x), None)
x = x + fc2(gelu(fc1(final_layer_norm(x))))
```

Pre-norm (rather than the original transformer’s post-norm) is what makes a 32-layer stack trainable without a warm-up schedule: the residual path from input to output is never normalised, so gradients reach the early layers intact. The `None` in the first line is the mask argument, and its absence is meaningful — encoder self-attention is **bidirectional**. Frame 40 may attend to frame 900. Nothing about audio requires causality, and forbidding it would throw away context for free. A final `layer_norm` closes the stack.

5 The decoder, and the asymmetry that defines large-v3-turbo

`TextDecoder` takes the tokens generated so far and the encoder’s 1500 acoustic frames, and returns logits over the vocabulary. Its layers add one component to the encoder’s recipe:

Masked self-attention (`self_attn`) over the tokens generated so far, with a **causal** mask so position i cannot see position $i + 1$. Without it, the training objective would be trivially satisfiable by reading the answer.

Cross-attention (`encoder_attn`) queries come from the text, keys and values from the **audio**. This is the only place the two modalities meet, and it is where “what does the recording say here” is actually answered.

Feed-forward (`fc1` / `fc2`) the same $1280 \rightarrow 5120 \rightarrow 1280$ block as the encoder, `gelu` in the middle.

Positions here are **learned** — `embed_positions` is a 448-entry table, sliced at `state.offset` so that appending a token reads the next row (§7). And the output projection is **tied**: rather than a separate `[d_model, vocab]` matrix, the logits are the token embedding used backwards,

```
x.matmul(embed_tokens.weight.transpose())
```

which is why the checkpoint has no `proj_out` tensor at all. Tying saves 66 million parameters and states an assumption worth knowing: the vector that **represents** a token and the vector that **predicts** it are the same thing.

5.1 32 against 4

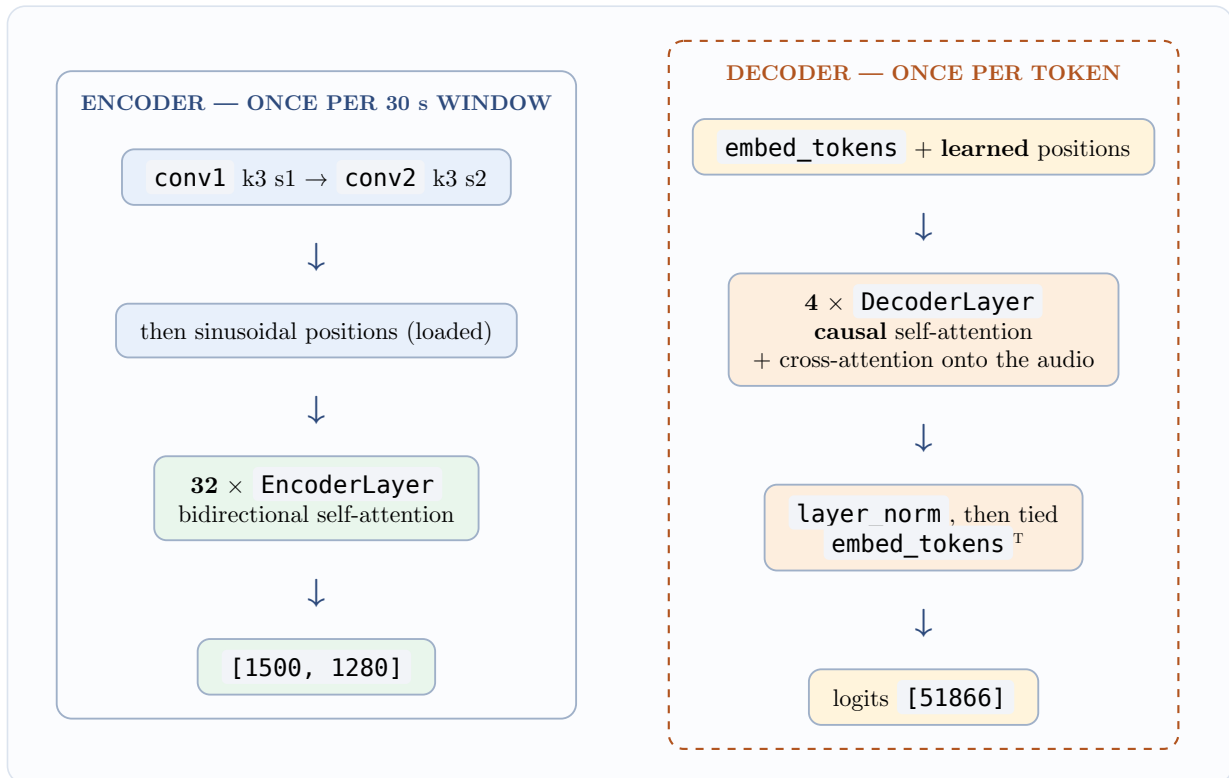


Figure 3: Where large-v3-turbo spends its compute. The encoder is untouched from large-v3; the decoder is distilled from 32 layers to 4. Because the encoder runs once per window and the decoder once per token, that asymmetry is worth far more than its parameter count suggests.

`whisper-large-v3-turbo` is `large-v3` with the decoder **distilled** from 32 layers down to 4 — the encoder is left alone. The trade is worth spelling out because it is the reason this is the default model (`hub_kit::DEFAULT_WHISPER`):

- The encoder’s cost is **fixed per 30 s of audio**, whatever comes out. The decoder’s cost is **per generated token**, and a dense 30 s segment can run to two hundred of them. Cutting the loop body by 8× is therefore an end-to-end speed-up of roughly the same order, while cutting encoder layers would buy almost nothing.
- What is lost is the decoder’s own linguistic capacity — its ability to carry long-range context and to reason about what the sentence is **likely** to say. In practice that shows up as slightly weaker recovery on genuinely ambiguous audio, and it is why translation quality degrades more than transcription quality. Transcription leans on cross-attention; translation leans on the language model.
- The vocabulary, the tokenizer and the special-token protocol are unchanged, so nothing downstream of the model has to know which size is loaded.

5.2 A note on dimensions (large-v3-turbo)

Every one of these is read from the checkpoint’s own `config.json` by `read_config`, never hardcoded — which is what makes switching to `large-v3`, or to a fine-tune, a download rather than a code change. `WhisperConfig` mirrors Hugging Face’s field names exactly, so a new size is a transcription rather than a derivation.

field	value	meaning
<code>num_mel_bins</code>	128	80 through large-v2; the front-end reads it from the model

<code>max_source_positions</code>	1500	encoder frames per 30 s window, after the stride-2 conv
<code>max_target_positions</code>	448	width of the decoder's learned position table
<code>d_model</code>	1280	residual-stream width, shared by both stacks
<code>encoder_layers</code>	32	runs once per window
<code>decoder_layers</code>	4	runs once per token — the distilled half
<code>encoder_attention_heads</code>	20	$d_{\text{head}} = 1280/20 = 64$
<code>decoder_attention_heads</code>	20	same head width
<code>encoder_ffn_dim</code> / <code>decoder_ffn_dim</code>	5120	$4 \times d_{\text{model}}$, the usual ratio
<code>vocab_size</code>	51866	BPE tokens plus the control tokens of §6
tensors in the checkpoint	587	what <code>load_safetensors</code> must apply, with 0 missing

5.3 Two details that only bite on a port

`k_proj` has no bias, while `q_proj`, `v_proj` and `out_proj` do. This is not an oversight upstream: a constant added to every key shifts all the logits in an attention row equally, and softmax is invariant to that, so the parameter would be dead weight. A port that adds the bias anyway will find no tensor for it in the checkpoint and fail loudly — which is the good outcome.

The reference scales **both** q and k by $d_{\text{head}}^{-1/4}$ rather than scaling the product by $d_{\text{head}}^{-1/2}$. Algebraically identical; the split exists as an fp16 overflow guard. `Attention::attend` is fp32 and applies the single $d_{\text{head}}^{-1/2}$ factor to q alone.

6 The special-token protocol: steering, not configuring

Whisper has no task flag, no language argument and no timestamp switch. It has a **prompt**. Everything the caller wants to control is expressed as **control tokens prepended to the sequence the decoder is asked to continue**, drawn from the same vocabulary as the text and carrying ids that Whisper reserved at the top of that vocabulary.

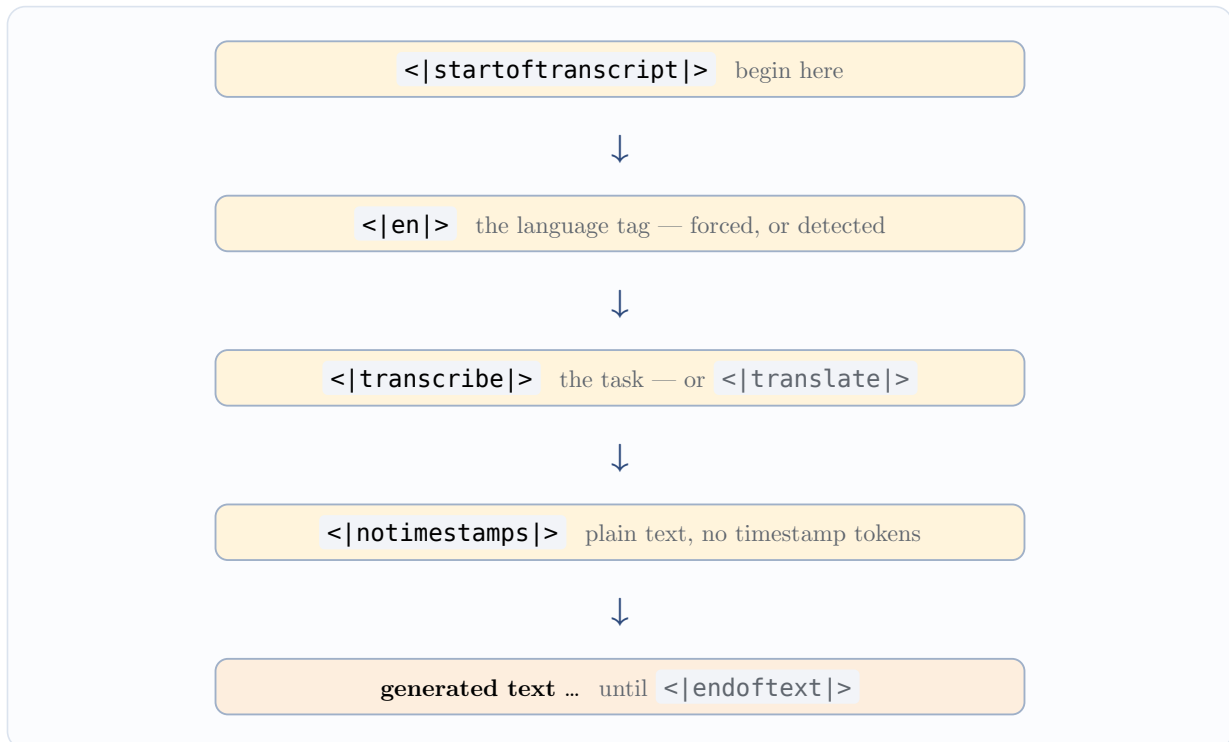


Figure 4: The prompt Whisper was trained on. The decoder is handed these four tokens and asked to continue; everything after them is transcript. The model is **steered** by the prefix, in exactly the way a language model is steered by its opening words.

<|startoftranscript|> (`Tokens::sot`) always first. On its own it is also the **language probe** — see below.

The **language tag** (`Tokens::languages` , keyed by ISO code) one of a hundred tokens. Supplying it forces the language; omitting it means detecting it.

<|transcribe|> or **<|translate|>** the task, held as `Tokens::transcribe` and `Tokens::translate` . Translation is **to English only** — Whisper has exactly one direction — which is why the toolkit’s `translate` stage is planned as a separate model rather than as a flag here.

<|notimestamps|> (`Tokens::no_timestamps`) suppresses the timestamp tokens Whisper can otherwise emit. The toolkit turns them off because segment boundaries come from the slicer upstream (§10), which knows where the silences actually are; taking timings from the model as well would be two sources of truth for one number.

<|endoftext|> (`Tokens::eot`) the model’s own stop signal. Generation ends when greedy search picks it.

Every one of these ids is read from the checkpoint’s `generation_config.json` , never written down. They move between releases — large-v3 added Cantonese and pushed all hundred language tokens up by one — and a hardcoded table would not fail on the wrong checkpoint, it would decode fluent nonsense. `Tokens::load` reads them; `Vocabulary` wraps the repo’s `tokenizer.json` for the BPE itself. The same file supplies two **suppression** sets: `suppress` , tokens never worth emitting anywhere (formatting marks, stray control tokens), and `suppress_at_start` , tokens forbidden only at the first generated position — a leading space, and an immediate **<|endoftext|>** that would produce an empty transcript.

Language detection (`detect_language`) exploits the protocol rather than adding machinery to it: feed **only** **<|startoftranscript|>** , take one decoder step, and read the logits at the language slots. The highest-scoring language token is the answer. It costs one step instead of a second full decode. The probe leaves a token in the cache, so `greedy` calls `Engine::restart` before the real prompt

goes in — a small thing, and exactly the kind of small thing that produces a transcript with one wrong word at the front.

7 Decoding: greedy search with a key/value cache

`stt-core`’s `greedy` is the loop, and it is short: prompt the decoder, take the `argmax` of the logits excluding the suppressed ids, stop on `<|endoftext|>`, otherwise append and step again.

7.1 Why a cache at all

Attention at step t needs the keys and values of **every** position up to t . Recomputing them from scratch each step means re-running the whole decoder over a sequence that grows by one every time — $O(T^2)$ work to produce T tokens, almost all of it identical to the previous step’s. Since the keys and values of past positions **cannot change** (that is precisely what the causal mask guarantees), they can simply be kept:

$$K_t = \begin{pmatrix} K_{t-1} \\ x_t W_k \end{pmatrix}, \quad V_t = \begin{pmatrix} V_{t-1} \\ x_t W_v \end{pmatrix}.$$

Each step then projects one token, concatenates, and attends — $O(T)$ total for the projections, with a single query row against a growing key matrix. That is `Attention::forward_cached`, and `KvCache` is the pair it appends to.

Cross-attention is cached differently, and matters more. The encoder output `xa` does not change across steps at all, so its keys and values are computed once on the first call and reused verbatim (`Attention::forward_cross`, via `get_or_insert_with`). Since `xa` is 1500 frames wide against a self-attention cache of a few dozen tokens, this is the larger saving of the two by a wide margin — and it is why the first decoder call of a window is visibly the expensive one.

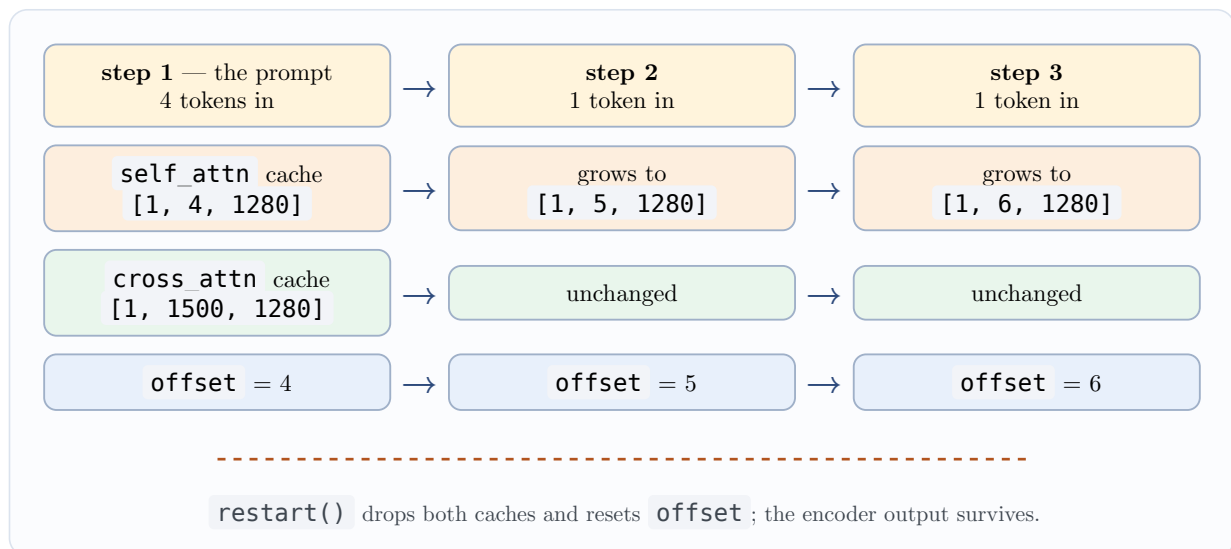


Figure 5: What `DecodeState` holds, and how it grows. Self-attention caches gain a row per step; cross-attention caches are computed once from the 1500-frame encoder output and then frozen. `offset` is what makes the learned positional table read the right row.

7.2 Where the cache lives, and why

`DecodeState` is held **outside** the `Module`, not as a field inside it. A `Burn Module` is cloned per device and written to checkpoints, and neither operation should be dragging a decode in progress along with it. One level up, in `stt-core`, the same reasoning goes further: the cache and the encoded audio stay **inside** the `Engine` implementation and never cross the trait boundary, because they are

backend-specific tensors with no useful common type — a `Tensor<Cuda, 3>` and an ONNX Runtime `Value` have nothing to say to each other, and hoisting them would mean copying to host memory every step for nothing. §9 takes that up.

7.3 Bounding the loop

`DecodeOptions::max_tokens` caps generation at 224 by default — half the 448-wide positional table, matching the reference. It exists for two different reasons at once, which is why hitting it is **reported** rather than treated as a normal ending: a dense 30 s segment can legitimately need more tokens, and a hallucination loop needs to be stopped. `Decoded::truncated` distinguishes “stopped at `<|endoftext|>`” from “ran out of budget”, and `Transcriber::window` logs a warning naming both fixes — raise the cap, or shorten the maximum clip length.

8 The causal-mask trap

This section is the single most valuable maintenance lesson in the repository. It cost a working port, it produced no error of any kind, and every automated check in place at the time passed.

8.1 What happened

The Burn port of Whisper loaded `openai/whisper-large-v3-turbo` at **587 tensors applied, 0 missing, 0 unused**. Perfect coverage. It then transcribed audio into fluent, confident, completely wrong text. The cause was one function call.

Burn’s `Tensor::triu_mask` and `Tensor::tril_mask` are named for the triangle they **keep**, not the one they mask out. The name `triu_mask` reads like “mask the upper triangle” — which is exactly what a causal mask must do — and it does the opposite. The correct call is:

`Tensor::tril_mask([n_q, n_kv], (n_kv - n_q) as i64, device)`

where the offset places the diagonal correctly when a cache of $n_{kv} - n_q$ earlier tokens is already present: query i sits at absolute position $n_{kv} - n_q + i$. That is `burn_whisper::causal_mask`, and the comment above it exists to stop the next reader from “fixing” it.

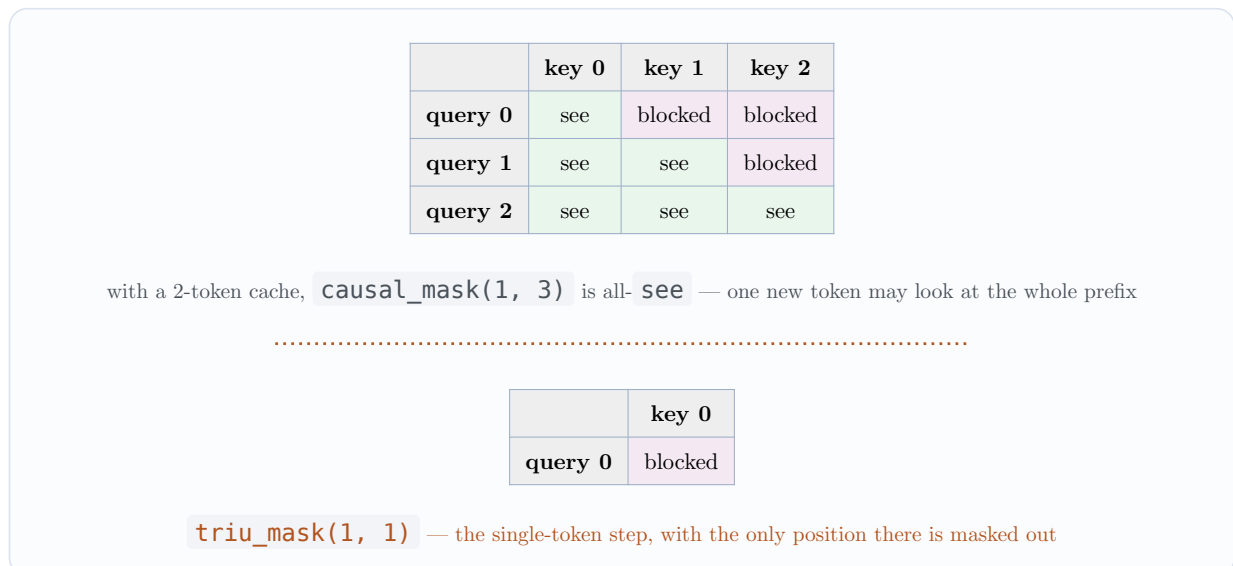


Figure 6: The mask for `causal_mask(3, 3)`, the full-prompt case. Shaded cells are blocked. `triu_mask` produces the mirror image of this — which does not crash, does not warn, and reverses the arrow of time.

8.2 Why it was silent, twice over

The failure survived because **two** independent things swallowed it.

- A fully-masked softmax row is **NaN**, not an error. During incremental decoding a step has one query and one key. With the mask reversed, that single position is blocked, so the attention row is a whole row of $-\infty$. Softmax of that is $e^{-\infty} / \sum e^{-\infty} = 0/0$ — **NaN** in every slot, propagated to every logit downstream, with no exception raised anywhere in the stack.
- Burn’s **assert_approx_eq** compares **NaN** against **NaN** without complaint. So a test that decodes one way and another and compares the two passes happily when **both** sides are entirely **NaN**. The test was not weak; it was measuring the wrong thing.

The lesson generalises past this one function: **a numerical test must assert finiteness explicitly, before it asserts agreement.** `burn-whisper`’s `incremental_decoding_matches_one_shot` does exactly that — it checks `x.is_finite()` over both tensors first, and only then compares — and `the_causal_mask_blocks_the_future_and_nothing_else` pins the mask itself against a literal truth table, including the $n_q = n_{kv} = 1$ case that produced the row of $-\infty$.

8.3 And why coverage did not catch it

Weight coverage answers one question: **does the module tree have a slot for every tensor in the checkpoint, and a tensor for every slot?** It is a statement about **layout**. The mask is not a parameter. No amount of coverage can say anything about it, and the same is true of a wrong activation, a transposed matmul, a scale factor off by $\sqrt{d}$, or a positional offset read one row late. This is why §11 exists.

9 Two runtimes, one loop

`stt-core`’s **Engine** trait is the entire boundary between the decode loop and a runtime, and it is four methods wide:

method	contract
<code>mel_bins()</code>	how many mel bands this model wants — the front-end reads it from the model rather than assuming 128
<code>encode(mel, frames)</code>	encode one window and begin a fresh decode against it
<code>step(tokens)</code>	feed tokens, return <code>Vec<f32></code> logits for the final position
<code>restart()</code>	drop the decode state, keep the encoded audio

Token ids in, `f32` logits out. That is all two very different runtimes can usefully agree on, and deliberately so: it keeps **Transcriber** **non-generic**, so picking a backend is a constructor call (`Transcriber::libtorch`, `Transcriber::cuda`, `Transcriber::wgpu`, `Transcriber::onnx`) rather than a type parameter that would infect every caller, and `stt-cli` never names a Burn type.

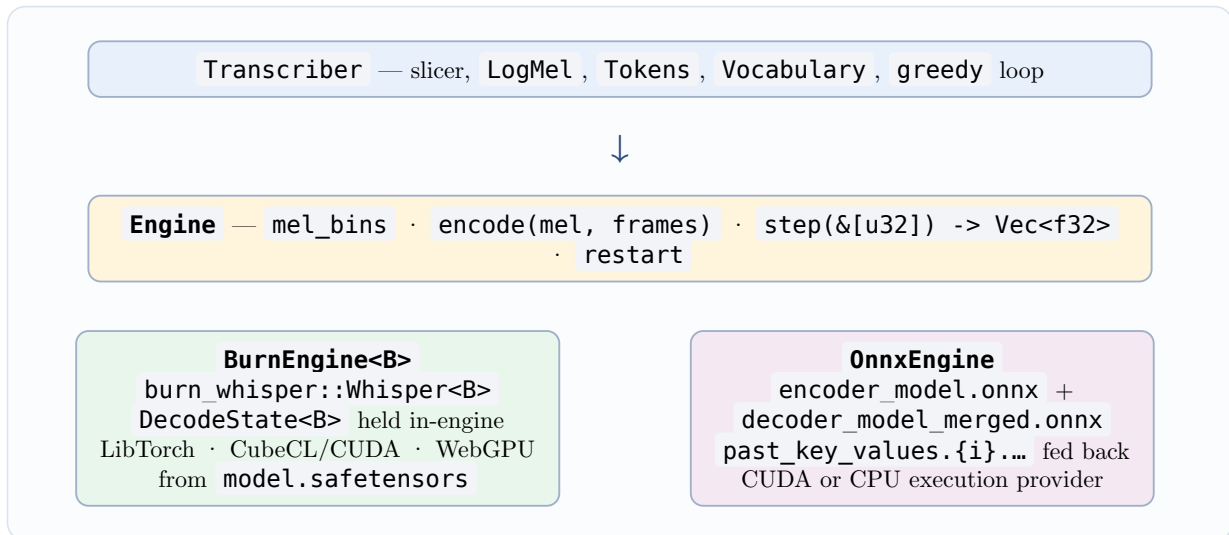


Figure 7: The `Engine` boundary. Everything above it is one code path; everything below is runtime-specific and never escapes. Both paths are first-class — the choice is made by what is in the model directory, then by what hardware is present.

9.1 The ONNX path is a deployment target, not a legacy

The ONNX Runtime engine consumes the two-graph export that `optimum` produces and `onnx-community` publishes. Most of `onnx_engine.rs` is that graph’s contract, and it is worth reading once because it is not obvious:

- `decoder_model_merged.onnx` folds the first (uncached) pass and the cached passes into **one** graph, selected by a boolean `use_cache_branch` input.
- For every layer the graph takes four past inputs and returns four matching outputs:

```
past_key_values.{i}.{decoder,encoder}.{key,value} -> present.{i}...
```

The `decoder` entries are self-attention and grow by a position per step; the `encoder` entries are the cross-attention over audio, computed on the first pass and then reused verbatim — the same optimisation `Attention::forward_cross` makes on the Burn side.

- Every past input must be **present** on the first pass even though the graph ignores it. Zero-length tensors stand in, built with `Tensor::new` rather than `from_array` — the latter validates raw data and rejects any zero dimension, which is exactly the shape wanted.

`SttBackend::Auto` resolves **weights first, hardware second**: `is_onnx_export` looks for an `encoder_model.onnx` and, finding one, selects ONNX Runtime; otherwise `burn_kit::auto_backend` picks LibTorch, then CubeCL/CUDA, then WebGPU. Naming a backend explicitly that is not compiled in is an error with a reason — only `auto` ever substitutes. Neither path is a fallback for the other: an ONNX export cannot run on Burn and safetensors cannot run on ONNX Runtime, so the files genuinely decide before preference does.

10 Segmentation, and hallucination on silence

Return to §1’s framing. Whisper is a language model conditioned on audio, and a language model’s job is to continue. Hand it thirty seconds of room tone and it will continue — with a plausible, well-punctuated sentence that nobody said. This is not a bug to be patched out of the decoder; it is the training objective working as specified on an input the objective never covered.

Three defences are in place, in increasing order of importance.

The suppression sets `Tokens::suppress` and `Tokens::suppress_at_start`, applied inside `argmax_excluding`. These stop the model emitting formatting junk and stop it ending instantly, but they say nothing about content.

The token cap `max_tokens` bounds what a runaway loop costs, and `truncated` makes it visible. A bound, not a cure.

Pre-segmentation don't hand the model silence in the first place. This is the one that works.

10.1 Reusing the sentence-safe slicer

`stt` segments with `audio_kit::slice` — the **same** slicer as the training preprocessing pass, with the same defence built in. The invariant that matters is stated in that module and is worth repeating here: **energy is used solely to locate long silent gaps between sentences, never to gate quiet-but-present sound**. A soft, breathy passage is low-energy and wanted; a between-sentence gap is low-energy and not. Distinguishing them is done by **duration**, not by level:

- Two voiced runs are treated as one sentence unless the silence between them is **both** below `silence_db` and at least `min_silence` long (`merged_voiced_runs`). Short internal pauses stay inside the clip, so a sentence is never split down the middle.
- Each kept segment is edge-padded by up to `pad` seconds of the bordering quiet, so onsets and breathy tails survive the cut, with the padding hard-bounded at the midpoint of the gap so neighbours can never overlap.
- Segments shorter than `min_clip` are dropped.

The one difference from the training path is `max_clip`. There it defaults to `0`, meaning **never split** — the trainer would rather have a long clip than a severed sentence. Here it is a hard 30 seconds, because **one segment must fit one encoder window**, and `stt-cli` rejects anything outside $(0, 30]$ before a model is even loaded. Over-long runs are split at their quietest interior frame (`split_long_ranges`, `quietest_cut`), with both halves constrained to stay above `min_clip` so splitting never manufactures a fragment the later filter would silently discard.

Everything after that is bookkeeping: `Transcriber::transcribe` walks the spans, transcribes each as one window, drops the empty ones, and reports each survivor as a `Segment` carrying start and end in seconds and the language actually used. `--format text` prints one line per segment so the output pipes straight into the next stage; `--format jsonl` adds the timings and the language, which is what a subtitle file or a synthesis training manifest needs.

One consequence of §3 is worth restating here, because it is the reason segmentation helps **twice**: mel normalisation is per-window and per-peak, so putting a loud passage and a whispered one in separate windows means the whisper is normalised against itself rather than against the loud passage.

11 Verifying the port

Weight coverage proves layout. §8 is the standing proof that layout is not enough. Every ported network in this repository therefore carries a **second** check that exercises arithmetic, and Whisper's is the strongest of them because it comes for free from having two runtimes:

Coverage — `examples/load` loads a real published checkpoint and reports applied / missing / unused. For `openai/whisper-large-v3-turbo` the answer must be **587 / 0 / 0**, and it must be **identical on every backend** — a backend that changes the counts is a bug. It also prints a handful of named parameters (`encoder.layer_norm.gamma` and friends) so their values can be diffed against the file by hand, because a matched tensor is not the same as a tensor that landed in the right slot.

Arithmetic — the **Burn-versus-ONNX transcript diff** run the same weights through `BurnEngine` and through `OnnxEngine` on the same audio and compare the text. They come out **byte-identical**. That is the check that catches everything coverage cannot — a reversed

mask, a wrong scale, a positional offset off by one, a transposed projection — because the two implementations share no code, only the checkpoint. An independent implementation is the best oracle available short of listening, and the toolkit happens to ship one for reasons that have nothing to do with testing.

Shape and cache invariants — the unit tests `encoder_halves_the_frame_rate` pins the stride-2 stem, `incremental_decoding_matches_one_shot` pins the mask and positional offset under a growing cache (asserting finiteness `first`), and `the_causal_mask_blocks_the_future_and_nothing_else` pins the mask against a literal table. These run without weights, which is what makes them worth having in CI.

If you change anything in `burn-whisper`, the sequence is: run `examples/load` and confirm 587 / 0 / 0 on at least two backends, run `cargo test -p burn-whisper`, then transcribe a real file on both `--backend tch` and `--backend onnx` and diff the output. The last step is the one that matters, and it is the one a green test suite will not do for you.

12 Recap: the whole path

Audio arrives as mono `f32` at 16 kHz. The slicer cuts it on genuine silence into pieces that each fit one encoder window. Each piece becomes a `[128, 3000]` log-mel picture, normalised against its own peak. The encoder — two convolutions that halve the frame rate, then 32 bidirectional transformer layers — turns that into 1500 acoustic frames, **once**. The decoder is primed with four control tokens that tell it where to start, what language to expect, whether to transcribe or translate, and not to emit timestamps; then it generates one token at a time, each step attending causally over what it has already said and by cross-attention over the frozen audio, carrying a key/value cache so the work per step stays constant. Greedy `argmax` picks each token, suppression sets rule a few out, `<|endoftext|>` ends it, and the BPE vocabulary turns the ids back into text.

The asymmetry is the design: analysis is expensive and happens once, generation is cheap and happens often, so large-v3-turbo spends 32 layers on the first and 4 on the second. The risk is the same asymmetry seen from the other side — generation is a language model, and a language model always has something to say. Segmenting the input is what keeps it answering the audio rather than itself.

Concepts anchored to the `voice` implementation: `burn-whisper` (the network), `stt-core` (front-end, tokenizer, decode loop, both engines), `stt-cli` (the `stt` binary), `audio-kit` (the slicer). Default checkpoint `openai/whisper-large-v3-turbo`, loaded unchanged from the first-party Hugging Face repo at 587 / 0 / 0.